learn-dev

Plateforme Web d'apprentissage de la programmation

Eric Bouchut – Titre professionnel Développeur Web et Web Mobile (DWWM)

Soutenance – RNCP 37674 – session août 2026

« *Apprendre la programmation, étape par étape* »

Introduction

Le besoin

- **Contexte**: Apprendre à programmer suppose un **parcours structuré** :
  - du contenu organisé,
  - une progression lisible,
  - un espace où un formateur publie et fait évoluer ses cours.
- **Problème** : offrir un environnement multi-rôles simple, **sûr** et **accessible**.
- **Solution** :

> `learn-dev` *est une plateforme Web où des formateurs publient des cours composés de leçons en Markdown, et où des étudiants s'inscrivent et les suivent.*

Vue d'ensemble de learn-dev

Rôles

- **étudiant** (`STUDENT`)
- **formateur** (`INSTRUCTOR`)
- **administrateur** (`ADMIN`)

Authentification et Autorisations

- Pas de restriction pour créer un compte étudiant
- **Authentification obligatoire**
- **Autorisations** (`RBAC`) :

  > Une fois **authentifié**, un utilisateur n'a accès qu'aux fonctionnalités que lui accorde son rôle.

Etudiant

Rôle `STUDENT` (**par défaut** à la création d'un compte) :

- consultation du **catalogue** des cours,
- inscription à des **cours**,
- lecture des **leçons**.

Formateur

Rôle `INSTRUCTOR` :

- **création de contenu** (cours/leçons),
- cycle de vie de **publication** des cours et leçons,
- gestion des **inscriptions**.

Administrateur

Rôle `ADMIN` :

- gestion des comptes,
- modération des contenus.

Conception

Gestion de projet

- GitHub **Project**,
- **Issues**,
- **ADR**s (*Architecture Decision Records*),
- **Plans** de développement,
- Développement **incrémental** : (authentification, domaine cours, création de contenu, administration),
- Git: **Branche par fonctionnalité**,
- **Conventional Commits**,
- **Pull-Requests**,
- **YAGNI**.

Cahier des charges

Objectifs fonctionnels

- s'inscrire et suivre des cours,
- gérer et publier des cours,
- administrer.

Objectifs non fonctionnel

- sécurité,
- accessibilité (*RGAA*),
- responsive,
- tests,
- Intégration Continue (CI).

Personas

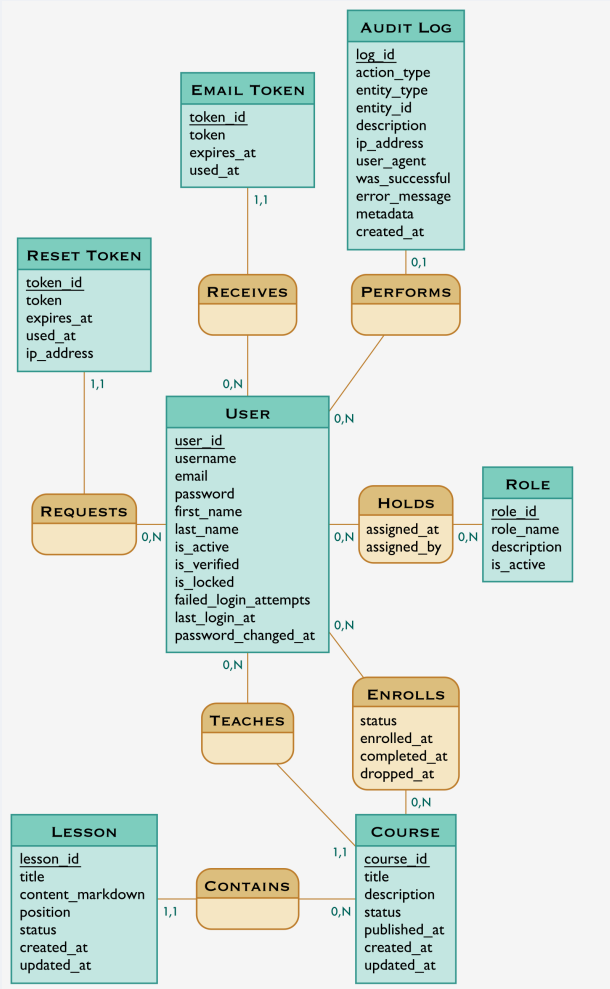
- **Léa** (étudiante)
  - s'inscrit et suit des cours, sur mobile comme sur ordinateur.
- **Marc** (formateur)
  - rédige les leçons de cours qu'il conçoit en Markdown,
  - gère la publication de ses cours,
- **Awa** (admin):
  - gère les comptes,
  - modère en toute sécurité.

Base de données (CP5)

Base de données – MCD

Modèle Conceptuel de Données

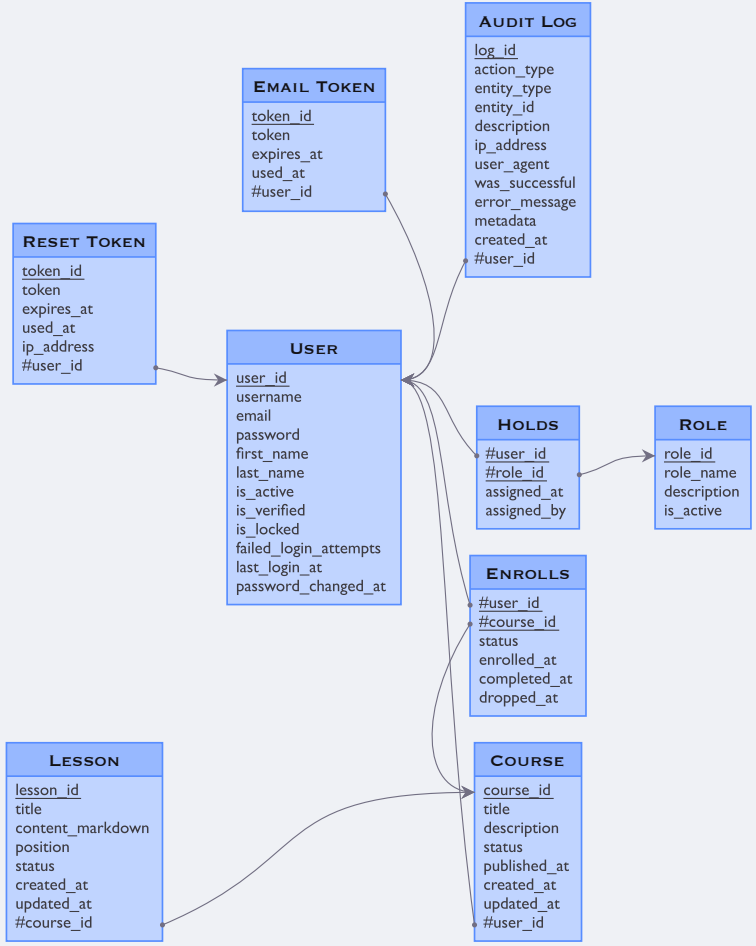
- Vue **client** de ses **donnés métier** :
  - **entités**
    - identifiants (soulignés)
  - **relations**
    - cardinalités
- Aucune syntaxe technique (base de données)



Base de données – MLD

Modèle Logique de Données

- **Indépendant de la base de données**
- **Structure relationnelle** partielle
  - **tables**
  - **tables de jointure**
    - modélisent la cardinalité `N,N`
  - **clé étrangères** (Foreign Keys)
    - précédées par `#`
    - modélisent les cardinalités `1,1` et `1,N`

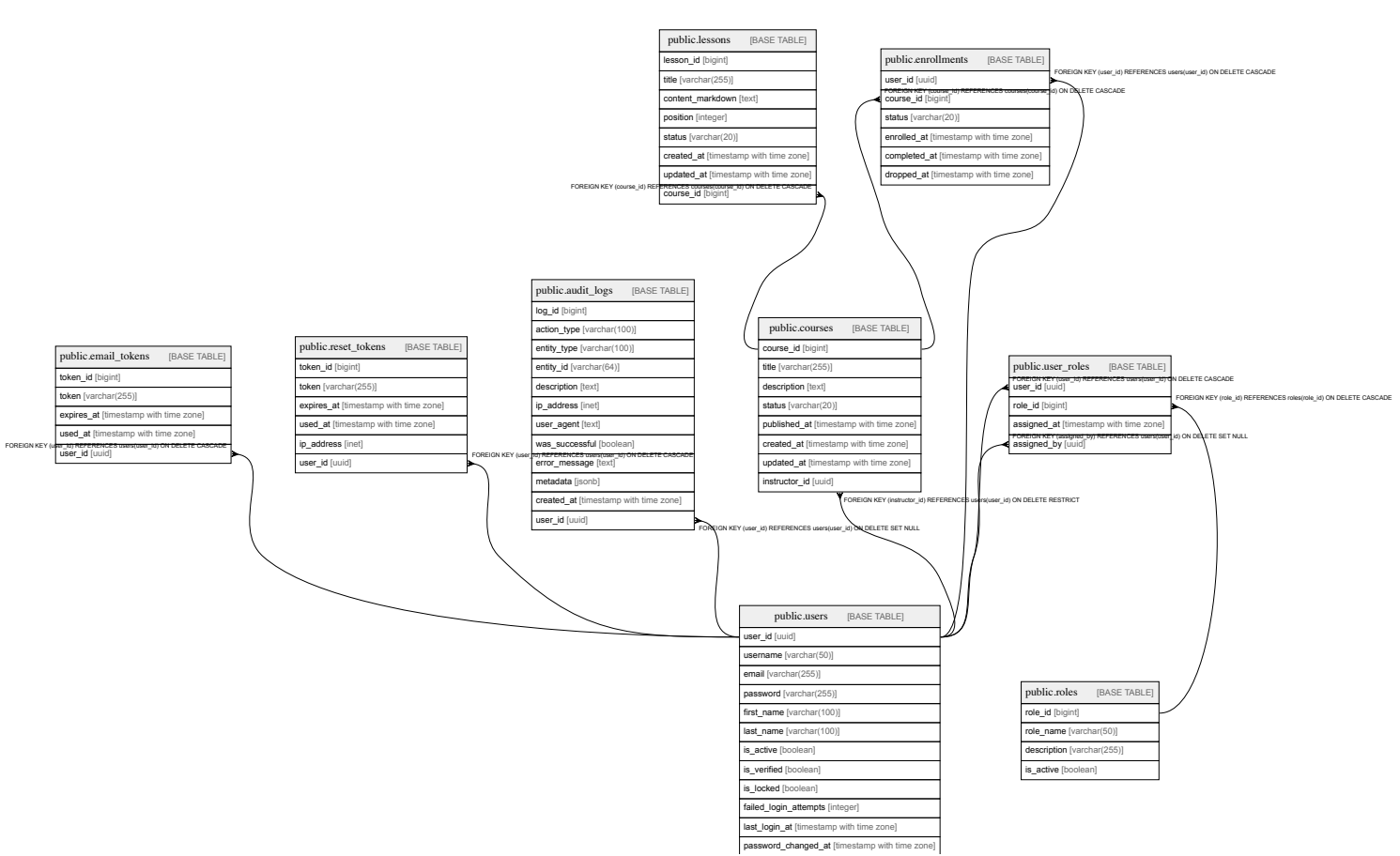


Base de données – MPD

Modèle Physique de Données

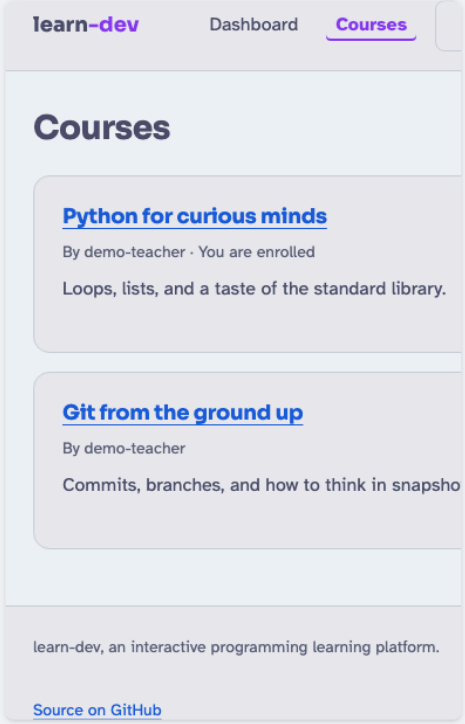
- Syntaxe spécifique au moteur de base de données relationelle (*SGBDR*)
- **PostgreSQL** (ADR-06),
- **Clés primaires** (choix d'architecture ADR-03)
  - `UUID` pour la table `users`
    - anti-IDOR
    - dans les 10 principales vulnérabilités **OWASP**
  - `BIGINT` pour les autres tables
  - **Clé primaire composite** pour les 2 tables de jonction

Diagramme MPD



Maquettage et accessibilité (CP2)

- **Maquettes Figma** + prototypes HTML ; charte, deux thèmes (Catppuccin / Soft Paper).
- **Responsive** : versions Web (bureau) **et Web mobile** (points de rupture en `rem`).
- **Accessibilité RGAA / WCAG AA** : structure sémantique, *skip link*, contrastes calculés (min. 4,73:1), police **Atkinson Hyperlegible**.



Environnement technique (CP1)

- **Java 21**
- **Maven** (dépendances, cycle de vie logiciel)
- **Spring Boot 3.5** (Web MVC, Data JPA, Validation, Mail)
- **Spring Security** (*RBAC*),
- **Thymeleaf** (templates/vues),
- **commonmark-java**: conversion du *Markdown* en *HTML*
- **jsoup**: assainissement des tags et attributs HTML
- **PostgreSQL 17** + **Liquibase**.
- **Docker** et **Docker Compose** (Postgres, Mongo, Mailpit)
- **Git**: Gestion de l'historique du code source
- **CI**: GitHub Actions.

Différences entre SQL et NoSQL

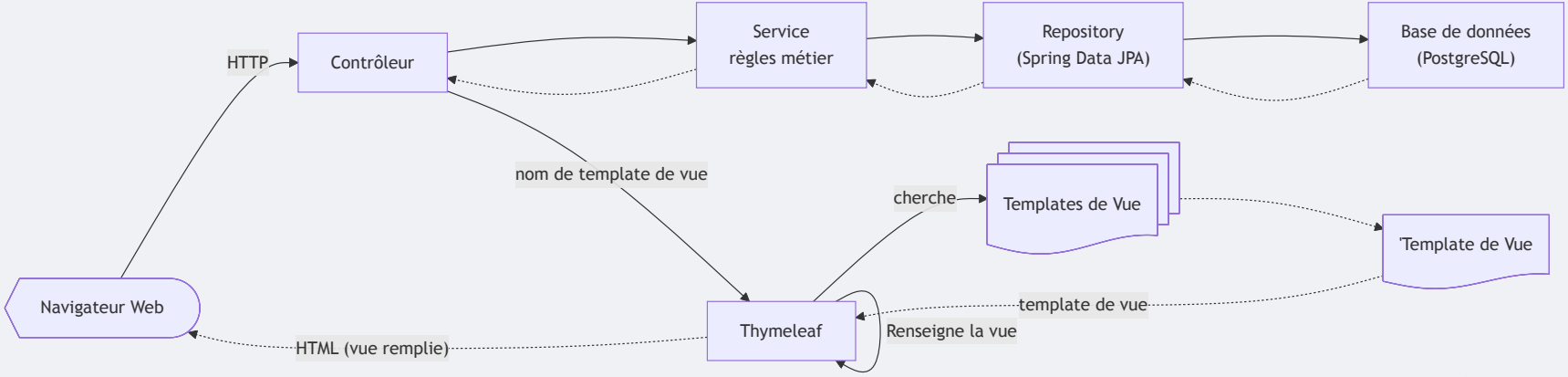
TODO

Développement

Architecture en couches

MVC (Controller Model View)

- `Controller` reçoit les requêtes HTTP entrantes et dispatche leur traitement et renvoie la réponse
  - `Service` applique les règles métier d'une entité.
    - `Repository` fait les interactions avec la base de données pour une entité.



Démo

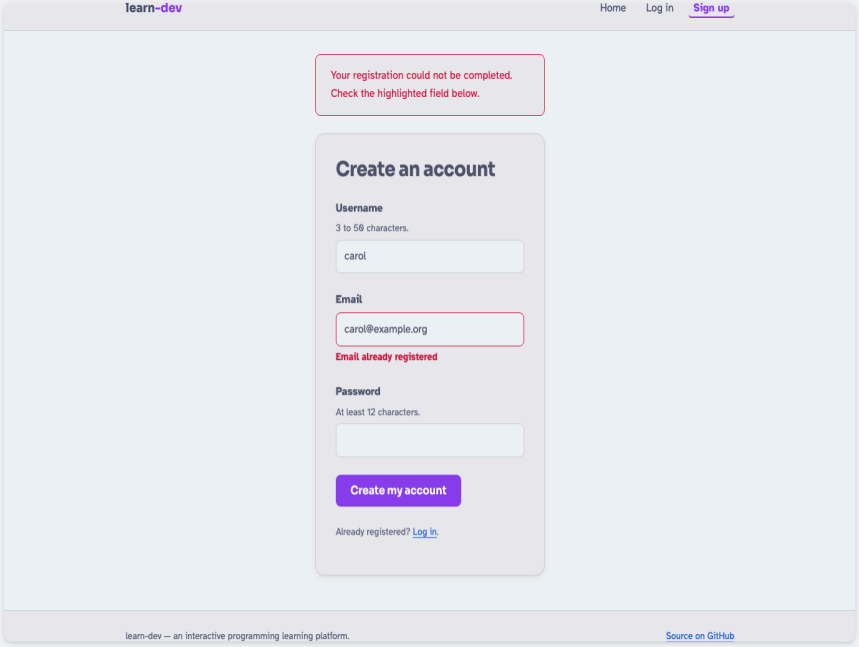
Deux fonctionnalités CRUD

- **Fonctionnalités**:
  1. **Inscription d'un utilisateur**
     - validation,
     - sécurité,
     - jeu d'essai.
  2. **Gestion des cours et leçons**
     - cycle de vie,
     - autorisations.
- **Erreurs HTTP** explicites :
  - `403` : Pas authentifié ou authorisation insuffisante
  - `404` : On ne le présente plus
  - `409` : conflit dans les paramètres de la requête

Feature 1 — Inscription d'un utilisateur (CP4, CP7)

- Formulaire **validé** (Bean Validation), **erreurs accessibles**.
- **Unicité** username/e-mail ; motif **Post-Redirect-Get**.

```
public User register(RegisterForm form) {
    if (users.existsByUsername(form.username())) {
        throw new DuplicateUsernameException();
    }
    if (users.existsByEmail(form.email())) {
        throw new DuplicateEmailException();
    }
    // hachage BCrypt, jamais de mot de passe en clair
    user.setPassword(encoder.encode(form.password()));
    return users.saveAndFlush(user);
}
```



Jeu d’essai — Inscription

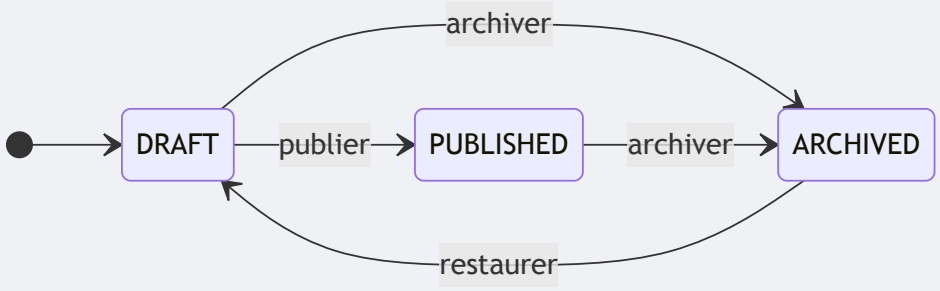
Scénario	Attendu	Obtenu
Cas nominal	Compte créé, mot de passe **haché**, rôle `STUDENT`	Conforme
E-mail déjà utilisé	Erreur de champ, **aucune** création	Conforme
Mot de passe trop court (< 8)	Rejet par la validation	Conforme
E-mail invalide	Rejet par la validation	Conforme

Feature 2 – Cours et leçons (CP7)

Espace formateur

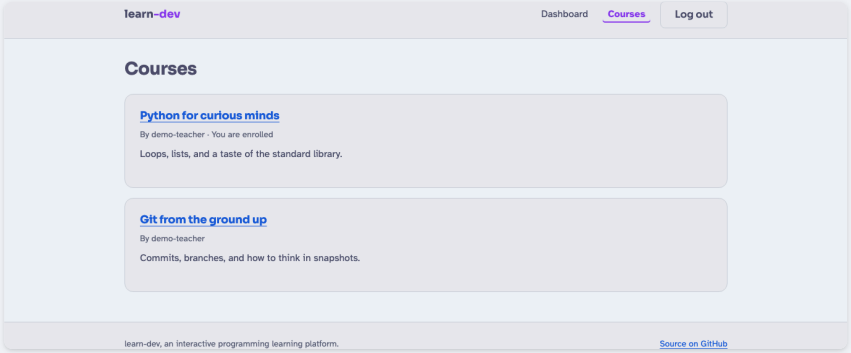
- création, mise à jour,
- publication,
- archivage/restauration,
- réordonnancement des leçons,
- gestion des inscrits
- **Autorisations** (erreurs *HTTP*):
  - cours d'un autre formateur => `403`,
  - contenu invisible => `404`,
  - transition invalide => `409`.

Cycle de vie



Front-end : interfaces et responsive (CP3, CP4)

- **21 gabarits Thymeleaf**, HTML5 sémantique, CSS responsive, fragment de mise en page partagé.
- Rendu **dynamique** du Markdown des leçons (converti puis **assaini** contre les XSS).
- Navigation précédent/suivant, affichage conditionnel selon le rôle.



Accès aux données (CP6)

- **Spring Data JPA** : *repositories* = interfaces, requêtes **dérivées** et **paramétrées** (anti-injection SQL) ; `@Transactional`.
- **NoSQL** : MongoDB **provisionné** en vue d'un stockage de contenu (non encore branché).

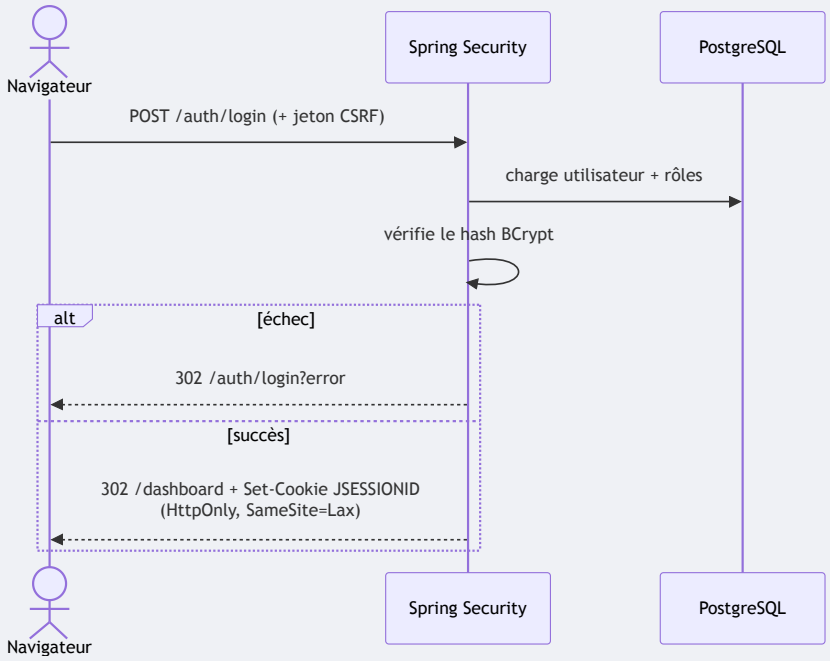
```java
public interface UserRepository extends JpaRepository<User, UUID> {

    @EntityGraph(attributePaths = "roles")   // l'utilisateur ET ses rôles en 1 requête
    Optional<User> findByUsername(String username);

    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}
```

Sécurité

- **BCrypt**, **CSRF** activé, **anti-XSS** (jsoup).
- **Anti-IDOR** (UUID), **anti-énumération**, **verrouillage de compte**, **rate-limiting**.
- Jetons **hachés SHA-256**, **journal d'audit**, matrice d'autorisations **testée**.
- Référence **OWASP Top 10** ; veille **Semgrep**.



Tests

- **64 tests** (pyramide) : unitaires (Mockito), tranche JPA, intégration (MockMvc).
- **PostgreSQL réel** en test via **Testcontainers** (schéma fidèle : UUID, contraintes).
- **Couverture** du code par les tests :
  - mesurée par **JaCoCo**,
  - publiée sur **Codecov**.

Déploiement

Conteneurisation et CI (CP8)

- **Image Docker multi-étapes** : build Maven puis exécution **non-root**.
- **Docker Compose** (Postgres / Mongo / Mailpit)
- **Profils** `dev` / `prod`
- **Secrets** via `.env`.
- **Procédure documentée**

```
docker compose up -d        # 1. services (base de données, SMTP local)
./mvnw spring-boot:run      # 2. application ⇒ http://localhost:8080
```

- **CI GitHub Actions** (4 workflows):
  - build,
  - tests,
  - lint,
  - contrôle anti-dérive du schéma.

Conclusion

Challenges

- concurrence à l'inscription (contrainte d'unicité captée),
- sécurité du rendu Markdown (assinissement des entrées pour éviter XSS),
- un seul `h1` par page (accessibilité).

Evolutions

- brancher **MongoDB** (NoSQL),
- API REST pour utiliser de React.
- suivre la progression leçon par leçon,

- **Dépôt** : github.com/ebouchut/learn-dev
- **Projet** : github.com/users/ebouchut/projects/7
- **Décisions d'architecture**: ADRs
- Palette/Thème:

Merci de votre attention.

Des questions ?